

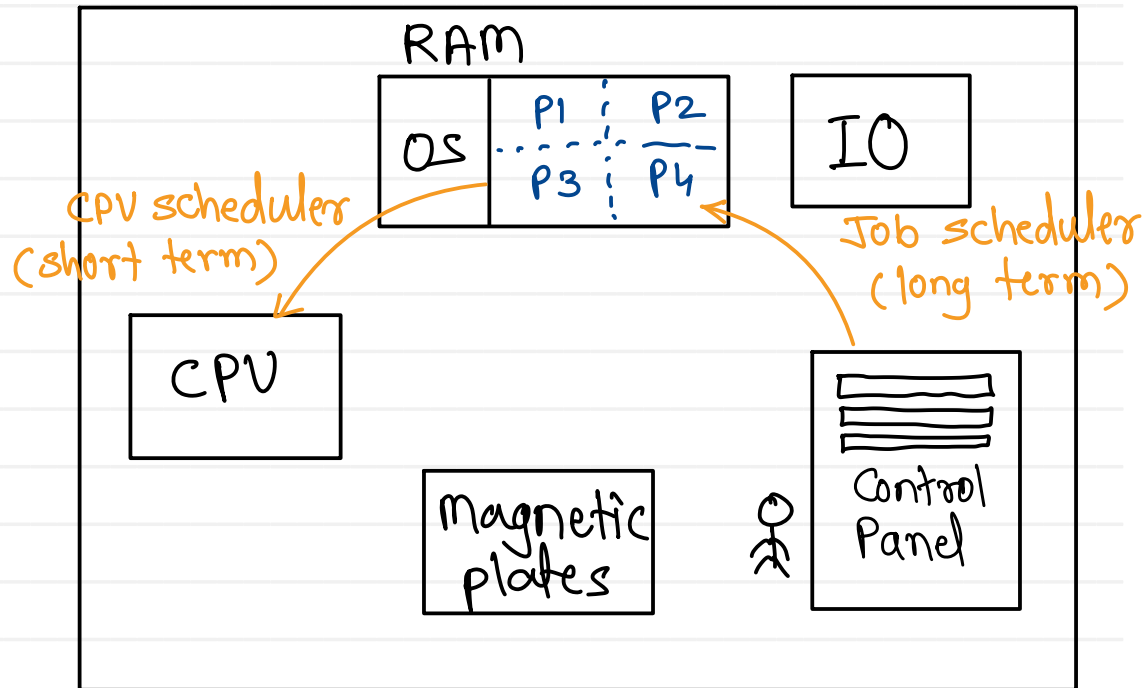


**Sunbeam Institute of Information Technology**  
**Pune and Karad**

## **Operating System Concepts and Linux Programming**

Trainer - Devendra Dhande

Email – [devendra.dhande@sunbeaminfo.com](mailto:devendra.dhande@sunbeaminfo.com)



1. Resident Monitor
2. Batch System
3. Multi programming system

- multiple programs are loaded into RAM

**CPU burst** : time spent on CPU by process

**IO burst** : time spent on IO by process

$\text{CPU burst} > \text{IO burst}$  : CPU bound process

$\text{IO burst} > \text{CPU burst}$  : IO bound process.

- mixture of CPU & IO bound process was loaded into RAM

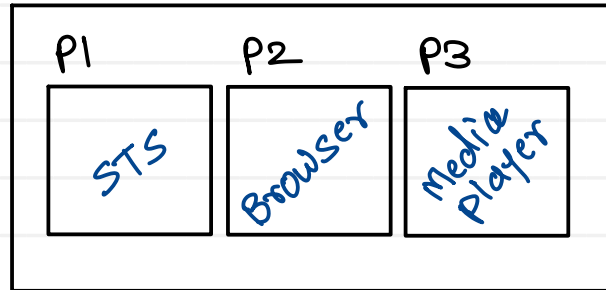
## 4. Time sharing system / Multitasking :

Response time  $< 1$  sec

- CPU time is shared in all the processes of RAM

There are two types of multitasking.

i. Process based Multitasking



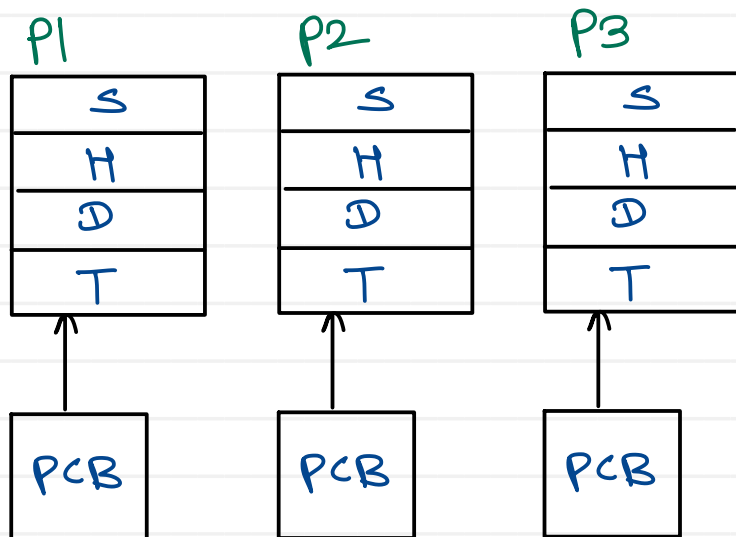
- system wide multitasking

ii. Thread based Multitasking  
(Multi Threading)



- Multitasking within process

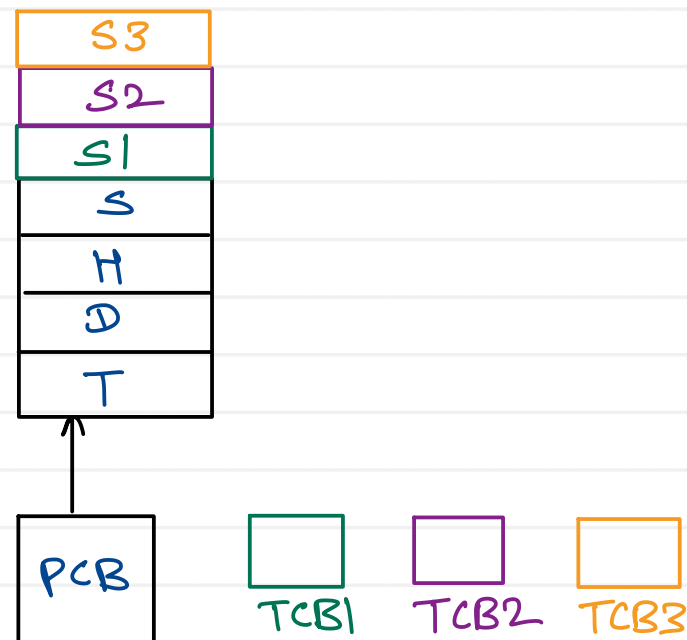
Processes are always created inside system



- Process is container of resources.
- Thread is alive entity which executes/run in container.
- every process has one default main thread.

Thread : light weight process

Threads are always created inside process.



## 5. Multiprocessing system:

- on single chip multiple CPUs are putted together, such process's are called as 'multi core' / 'multi processor'
- multiple processes can be executed parallelly.
- such systems are called as "multiprocessing systems".

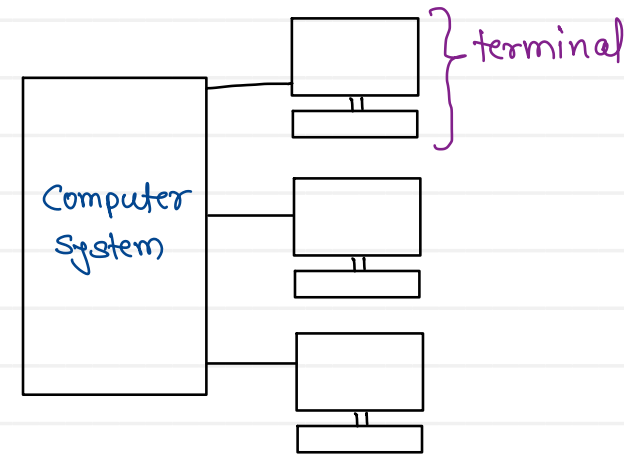
### 1. Symmetric Multiprocessing

OS treats all processor equally & schedules any process on any core (processor).

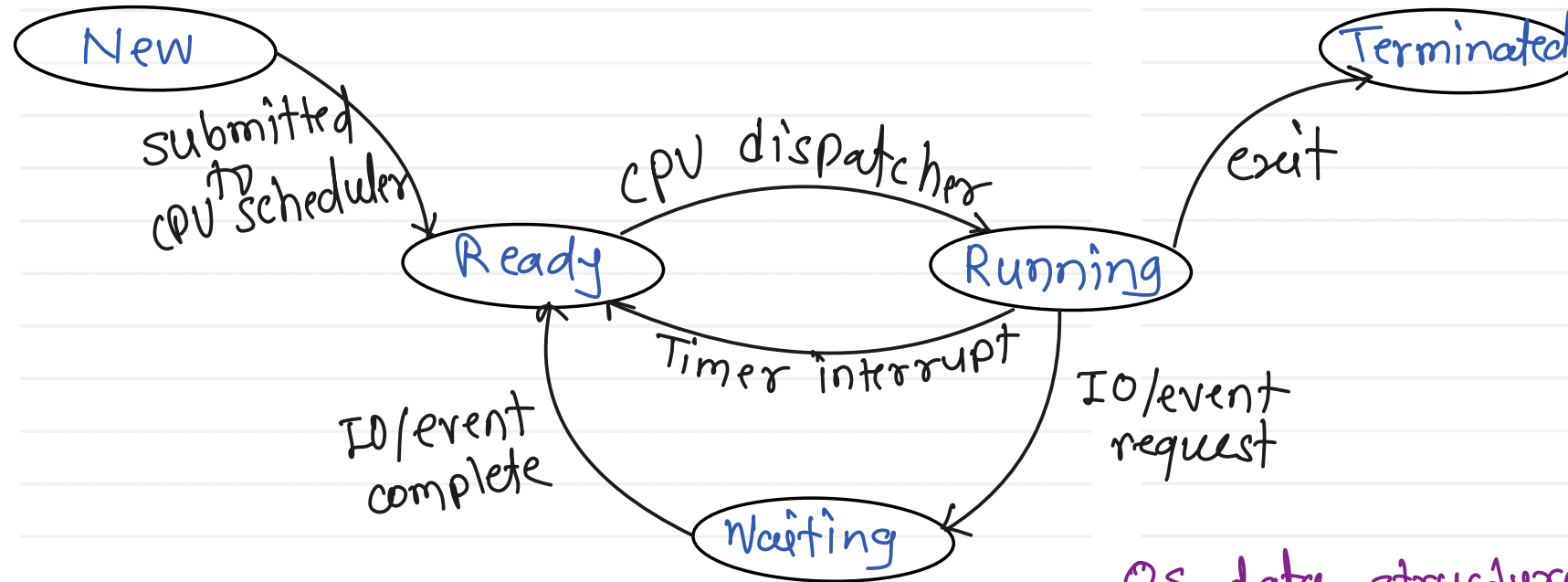
### 2. Asymmetric Multiprocessing

OS do not treat all cores equally. OS only scheduler process for one core & that core internally divides task into remaining cores.

## 6. Multiuser system



|     |          |        |
|-----|----------|--------|
| ps  | uname    | tz     |
| top | uname -r | whoami |
|     | uname -a | who    |
|     |          | N      |



## OS data structures:

1. Job queue / Process list :
  - all processes of system
2. Ready queue :
  - processes which are ready to execute on CPU
3. Waiting queues :
  - processes which are waiting for IO/event

CPU scheduler is called in below 4 cases:

1. Running → Terminated } voluntarily
2. Running → Waiting
3. Running → Ready } forcefully
4. Waiting → Ready

Scheduling Algorithms:

1. FCFS
2. SJF
3. Priority
4. RR

Types of scheduling

1. Non Preemptive

- CPU access is always given to the next process voluntarily
- high priority process may need to wait for low priority process

2. Preemptive

- CPU access can be given to the next process forcefully
- high priority process never waits for low priority process

# CPU scheduling criteria's

## 1. CPU Utilization : (Max)

- how much CPU is busy / idle e.
- e.g. CPU Utilization = 70%

Busy = 70% & Idle = 30%

Server OS - 90% CPU Utilization

Desktop OS - 70% CPU Utilization

## 2. Through put : (Max)

- amount of work done in unit time
- number of processes completed in unit time

$$TAT = \text{CPU waiting time} + \text{CPU burst} + \text{IO waiting time} + \text{IO burst}$$

## 3. Response time : (min)

- time from arrival of process into ready queue upto first time scheduled on CPU

$$RT = ST - AT$$

## 4. Waiting time : (min)

- total time spent by process in ready queue

$$WT = TAT - \text{CPU burst}$$

## 5. Turn Around Time (TAT) : (min)

- total time spent by process inside memory (RAM)
- time from arrival of process inside memory upto departure of process from memory.

$$TAT = DT - AT$$



## Execution context:

values of CPU registers

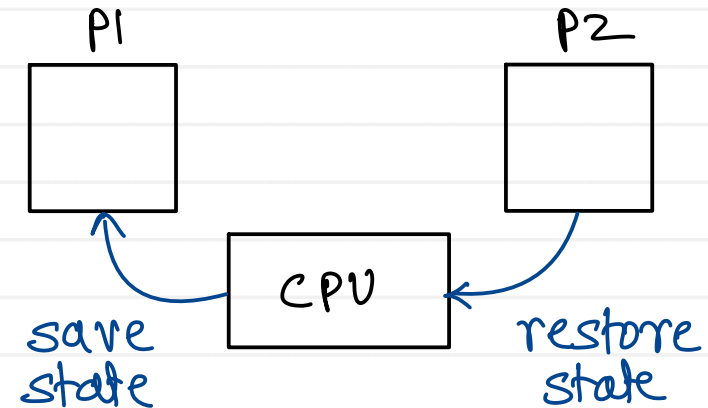
↑  
small memory element which are present inside CPU

1. General purpose registers - A, B, C, D..
2. Flag register
3. Segment registers - SS, DS, CS, ES----
4. Offset registers - SP, BP, SI, DI
5. Program Counter - PC/IR

- CPU dispatcher loads execution context of process from its PCB to CPU.

## Context switch:

- changing process of CPU.
- execution context of running process is saved first & then execution context of selected process is restored on CPU



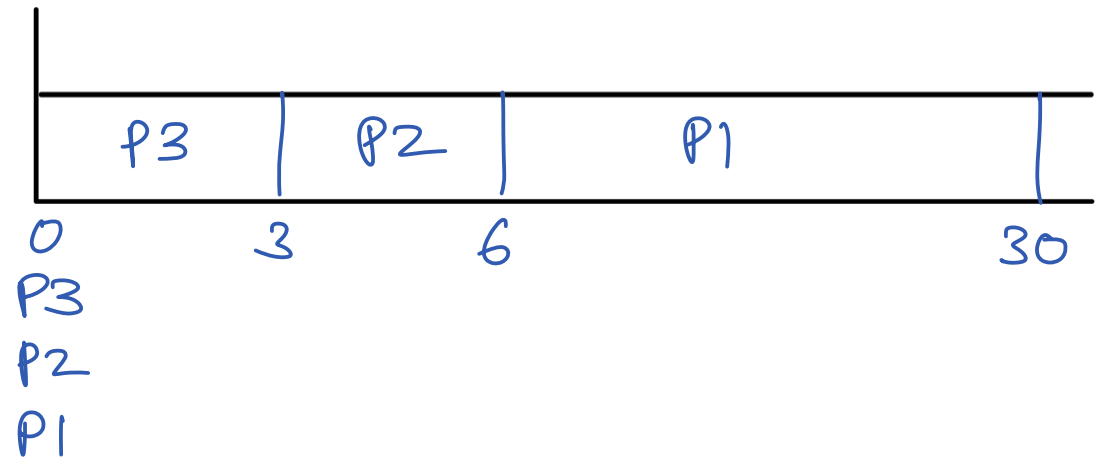
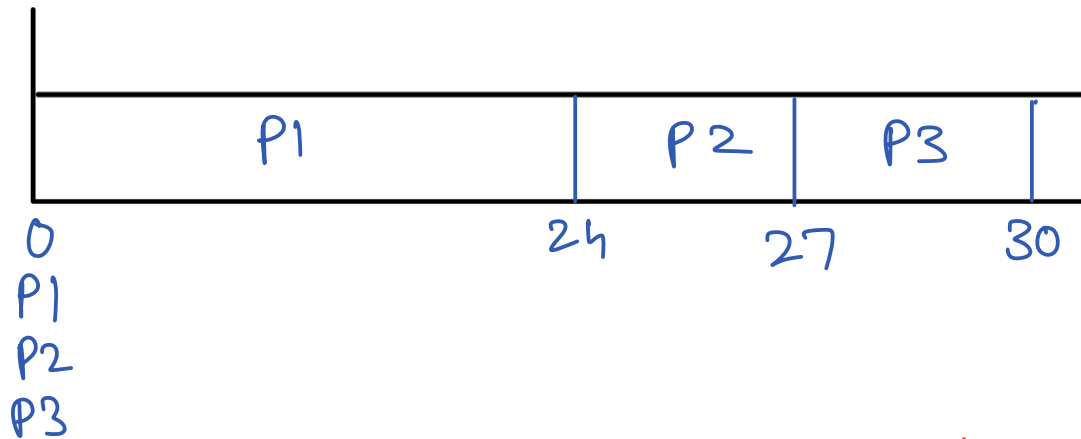
# FCFS (First Come First Serve) (Non preemptive)

| Process | Arrival | CPU Burst |
|---------|---------|-----------|
| P1      | 0       | 24        |
| P2      | 0       | 3         |
| P3      | 0       | 3         |

| RT | WT | TAT |
|----|----|-----|
| 0  | 0  | 24  |
| 24 | 24 | 27  |
| 27 | 27 | 30  |

| Process | Arrival | CPU Burst |
|---------|---------|-----------|
| P3      | 0       | 3         |
| P2      | 0       | 3         |
| P1      | 0       | 24        |

| RT | WT | TAT |
|----|----|-----|
| 0  | 0  | 3   |
| 3  | 3  | 6   |
| 6  | 6  | 30  |



Convoy effect:

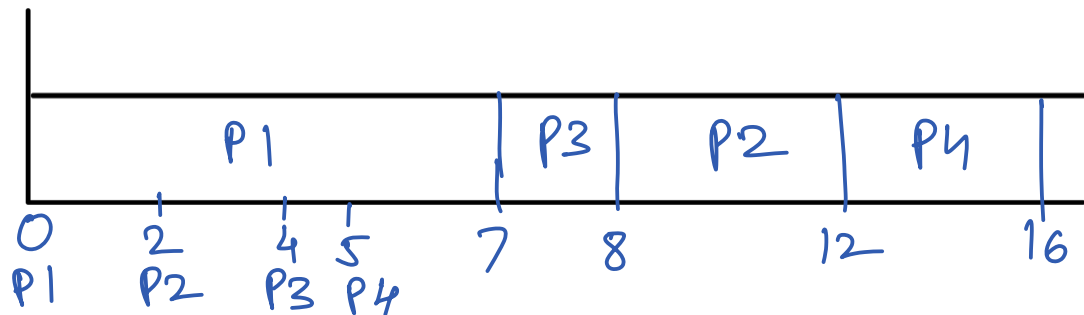
- due to arrival of longer process early, all other processes need to wait for longer duration

# SJF (Shortest Job First)

(Non Preemptive)

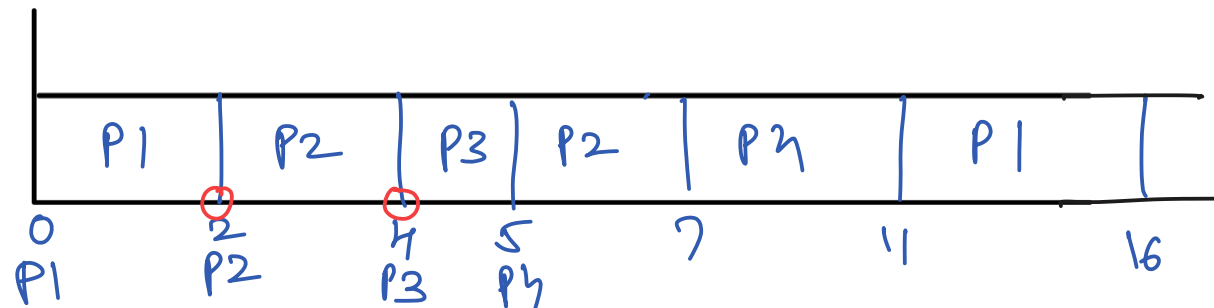
| Process | Arrival | CPU Burst |
|---------|---------|-----------|
| P1      | 0       | 7         |
| P2      | 2       | 4         |
| P3      | 4       | 1         |
| P4      | 5       | 4         |

| RT | WT | TAT |
|----|----|-----|
| 0  | 0  | 7   |
| 6  | 6  | 10  |
| 3  | 3  | 4   |
| 7  | 7  | 11  |



Shortest Remaining Time first  
(Preemptive)

| Process | Arrival | CPU Burst |
|---------|---------|-----------|
| P1      | 0       | 7         |
| P2      | 2       | 4         |
| P3      | 4       | 1         |
| P4      | 5       | 4         |



Starvation:

due to longer CPU burst process need to wait for longer duration inside ready queue to get CPU access.